

Next-Word Prediction with LSTMs in PyTorch

From a Text Corpus to Autoregressive Generation

Abstract

A next-word predictor estimates which vocabulary token should follow a sequence of earlier tokens. These notes build that task from first principles with a long short-term memory (LSTM) network in PyTorch. The complete path is made explicit: text normalization, tokenization, vocabulary construction, prefix–target expansion, padding, tensor creation, batching, embedding lookup, gated recurrence, cross-entropy training, device transfer, one-step prediction, and repeated autoregressive generation. The project configuration contains 289 vocabulary classes, 942 derived training rows, context width 61, embedding width 100, hidden width 150, and 223,739 trainable parameters. Every important tensor axis and state transition is defined. One scalar LSTM step is expanded numerically and matched exactly against `nn.LSTMCell`. Important corrections are integrated: omitted initial states are zeros rather than random vectors; `nn.LSTM` returns two top-level objects; padding and unknown tokens need distinct IDs; training loss is not evidence of general language ability; and an out-of-domain prompt can receive a confident but meaningless continuation. The result is a rigorous implementation guide rather than a memorized code recipe.

Contents

1	Purpose and system contract	2
1.1	What a next-word predictor computes	2
1.2	End-to-end data flow	3
2	The language-modeling objective	4
2.1	Conditional next-token probabilities	4
2.2	Creating labels from unlabelled text	4
3	Tokenization and vocabulary construction	4
3.1	Purpose of tokenization	4
3.2	Sentence splitting and punctuation	5
4	Prefix rows, padding, and the input–target split	6
4.1	Building every prefix	6
4.2	A complete numerical padding example	6
4.3	Why padding and unknown IDs must differ	7
5	Dataset and DataLoader contracts	7
5.1	Dataset responsibilities	7
6	Embedding lookup: token IDs become vectors	9
6.1	Table definition and shapes	9

7	LSTM state and gates	9
7.1	Why two recurrent states are used	9
7.2	A fully expanded scalar LSTM step	10
7.3	Unrolling through 61 context positions	12
8	The PyTorch architecture and tensor trace	12
8.1	Module responsibilities	12
8.2	Complete batch shape trace	13
8.3	Parameter count	14
8.4	The batch-axis failure	14
9	Training objective and optimizer update	15
9.1	Cross-entropy for one next-token class	15
9.2	Device-safe mini-batch loop	15
10	One-step prediction	16
10.1	Prediction must reproduce preprocessing	16
11	Autoregressive generation	17
11.1	Repeated one-step prediction	17
11.2	Top-k suggestions and sampling	19
12	Padding-aware production design	19
12.1	Right padding and true lengths	19
12.2	Why the saved artifact is more than model weights	20
13	Failure modes and disciplined repairs	20
13.1	Evaluation beyond training loss	20
13.2	Engineering improvements	21
14	Review questions and exercises	21

1 Purpose and system contract

1.1 What a next-word predictor computes

A **next-word predictor** receives a sequence of words and returns a candidate for the following word. For example, a model may map the context **how are** to **you**. Mobile keyboards, e-mail composition tools, search boxes, and writing assistants use related ideas to offer context-sensitive suggestions.

Definition

A **language model** assigns a probability distribution to possible continuations of a token sequence. A **token** is one discrete text unit, such as a word or punctuation mark. The **context** is the sequence already available when the next token is predicted.

The application in these notes uses a small frequently-asked-questions corpus. It can reproduce local patterns such as

- you will have to fill → a google form provided ...;
- the course follows a monthly → subscription model where you have to make

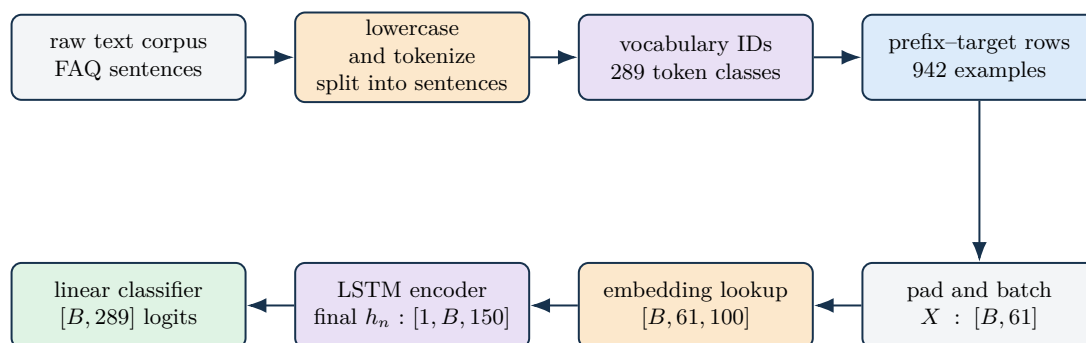
These continuations illustrate learned corpus patterns. They do not establish grammatical understanding, factual reliability, or performance on unrelated text.

1.2 End-to-end data flow

The complete training path is

raw text → tokens → vocabulary IDs → prefix–target rows → padded batches → LSTM logits.

A **vocabulary ID** is an arbitrary integer label for one token. A **prefix–target row** pairs earlier tokens with the immediately following token. A **logit** is an unrestricted real-valued class score; it is not yet a probability.



At prediction time, the largest logit selects one next token. Appending that token to the context and repeating the same computation creates autoregressive text generation.

Figure 1: The complete next-word system turns a text corpus into supervised rows, maps IDs through an LSTM, and emits one vocabulary logit per possible next token. [Technical reference](#).

Four implementation stages keep responsibilities separate:

1. prepare tokens, vocabulary IDs, prefixes, targets, and padding;
2. define the embedding, LSTM, and linear classifier;
3. optimize all trainable parameters on mini-batches;
4. reuse identical preprocessing for one-step and repeated prediction.

Takeaway

The model learns conditional patterns present in one corpus. Repeated prediction turns a one-step classifier into a generator, but repetition does not expand the model's evidence beyond its training data.

2 The language-modeling objective

2.1 Conditional next-token probabilities

Let a token sequence be

$$w_1, w_2, \dots, w_T,$$

where w_t is the token at position t and T is the sequence length. At position t , the model estimates

$$P(w_{t+1} \mid w_1, \dots, w_t).$$

The vertical bar means “conditioned on.” The expression therefore asks for a distribution over the next token after observing the prefix through position t . This quantity matters because it supports both one-step suggestions and complete-sequence scoring.

The probability of a complete sequence can be factored with the chain rule:

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}).$$

Each factor scores one observed token using only the earlier prefix. The product explains why next-token training is a language-modeling objective rather than an unrelated classification trick.

2.2 Creating labels from unlabelled text

Raw text does not contain a separate target column. It creates targets from its own order. For the five-token sentence

`[python, libraries, for, data, science],`

the supervised rows are

row	input prefix	target
1	python	libraries
2	python libraries	for
3	python libraries for	data
4	python libraries for data	science

The construction grows the prefix by one token at each row. A sentence of n tokens contributes $n - 1$ rows because the first token has no nonempty earlier prefix in this scheme.

If the vocabulary IDs are

$$\text{python} \mapsto 1, \quad \text{libraries} \mapsto 2, \quad \text{for} \mapsto 3, \quad \text{data} \mapsto 4, \quad \text{science} \mapsto 5,$$

then the same rows are $[1] \rightarrow 2$, $[1, 2] \rightarrow 3$, $[1, 2, 3] \rightarrow 4$, and $[1, 2, 3, 4] \rightarrow 5$. The numbers identify tokens; their numerical magnitudes carry no semantic order.

3 Tokenization and vocabulary construction

3.1 Purpose of tokenization

Tokenization divides text into units that can receive vocabulary IDs. Lowercasing merges forms such as `Python` and `python`. The project uses NLTK’s `word_tokenize`, which requires the `punkt` and, in recent NLTK releases, `punkt_tab` data packages.

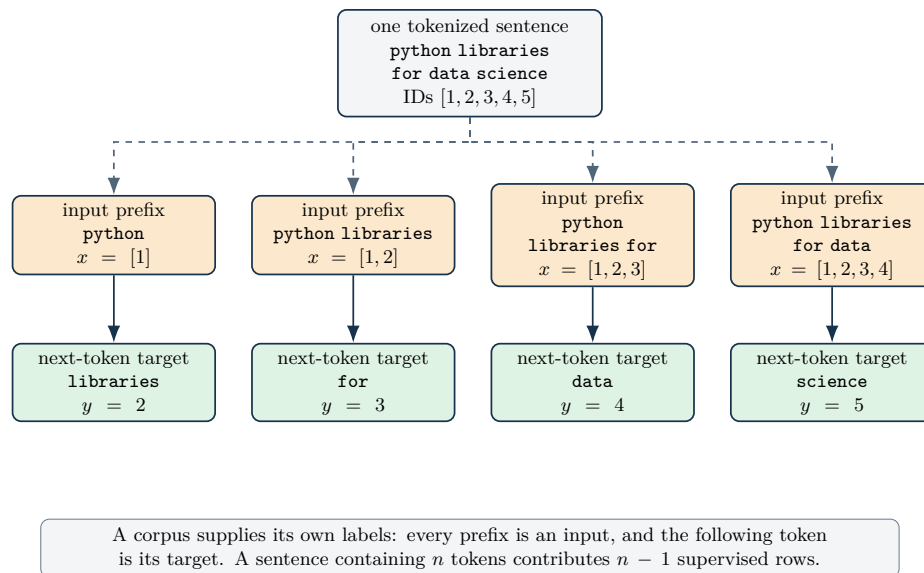


Figure 2: One five-token sentence becomes four supervised prefix–target examples, preserving token order and the immediately following label. [Technical reference](#).

```

from collections import Counter
from nltk.tokenize import word_tokenize

UNK_TOKEN = "<UNK>"

def tokenize(text):
    return word_tokenize(text.lower())

def build_vocab(document):
    tokens = tokenize(document)
    vocab = {UNK_TOKEN: 0}
    for token in Counter(tokens).keys():
        if token not in vocab:
            vocab[token] = len(vocab)
    return vocab

def text_to_indices(tokens, vocab):
    unknown_id = vocab[UNK_TOKEN]
    return [vocab.get(token, unknown_id) for token in tokens]
  
```

Listing 1: Tokenize text, build a vocabulary, and map tokens to integer IDs

A **vocabulary** is the dictionary from token strings to integer IDs. The prepared FAQ corpus contains 289 entries when the unknown token is included. The mapping must be saved with the model because output index 17 is meaningless without the same index-to-token table used during training.

3.2 Sentence splitting and punctuation

Splitting a document with `document.split(". ")` is easy to inspect but linguistically fragile. It treats every period as a boundary, loses that punctuation mark, and mishandles abbreviations, decimal numbers, URLs, and repeated newlines. NLTK tokenization also makes punctuation tokens explicit, so strings such as `799/month` may split differently from whitespace-only tokenization.

The data preparation reports 78 extracted sentence strings and 942 training sequences. A stray count of 79 is inconsistent with the reported list length and must not be mixed into tensor-shape reasoning. The tensor dimensions `[942, 62]`, `[942, 61]`, and `[942]` provide the executable contract used below.

Pitfall

The preprocessing function is part of the model. Changing lowercasing, sentence splitting, punctuation rules, or vocabulary order after training changes token IDs and invalidates learned embedding rows.

4 Prefix rows, padding, and the input–target split

4.1 Building every prefix

For each numerical sentence $s = [s_1, \dots, s_n]$, construct

$$[s_1, s_2], [s_1, s_2, s_3], \dots, [s_1, \dots, s_n].$$

Every sequence includes both its input prefix and final target. The following implementation makes that invariant explicit.

```
import torch

PAD_ID = 0

def build_training_tensors(numerical_sentences):
    sequences = []
    for sentence in numerical_sentences:
        for end in range(2, len(sentence) + 1):
            sequences.append(sentence[:end])

    max_length = max(len(sequence) for sequence in sequences)
    padded = [
        [PAD_ID] * (max_length - len(sequence)) + sequence
        for sequence in sequences
    ]
    padded = torch.tensor(padded, dtype=torch.long)
    inputs = padded[:, :-1]
    targets = padded[:, -1]
    return inputs, targets, max_length
```

Listing 2: Create prefix sequences, left-pad them, and split inputs from targets

The longest combined prefix-plus-target sequence has length 62. Therefore

$$\text{padded} : [942, 62], \quad X : [942, 61], \quad y : [942].$$

The first axis counts examples. The second axis of X is context position. Each scalar in y is one next-token class ID.

4.2 A complete numerical padding example

For IDs $[1, 2, 3, 4, 5]$, left-padding to length five gives

$$\begin{aligned} [1, 2] &\mapsto [0, 0, 0, 1, 2], \\ [1, 2, 3] &\mapsto [0, 0, 1, 2, 3], \\ [1, 2, 3, 4] &\mapsto [0, 1, 2, 3, 4], \\ [1, 2, 3, 4, 5] &\mapsto [1, 2, 3, 4, 5]. \end{aligned}$$

Removing the last column creates the four-token input rows; the removed values $[2, 3, 4, 5]$ become targets. The resulting toy shapes $X : [4, 4]$ and $y : [4]$ match the same operation used at corpus scale.

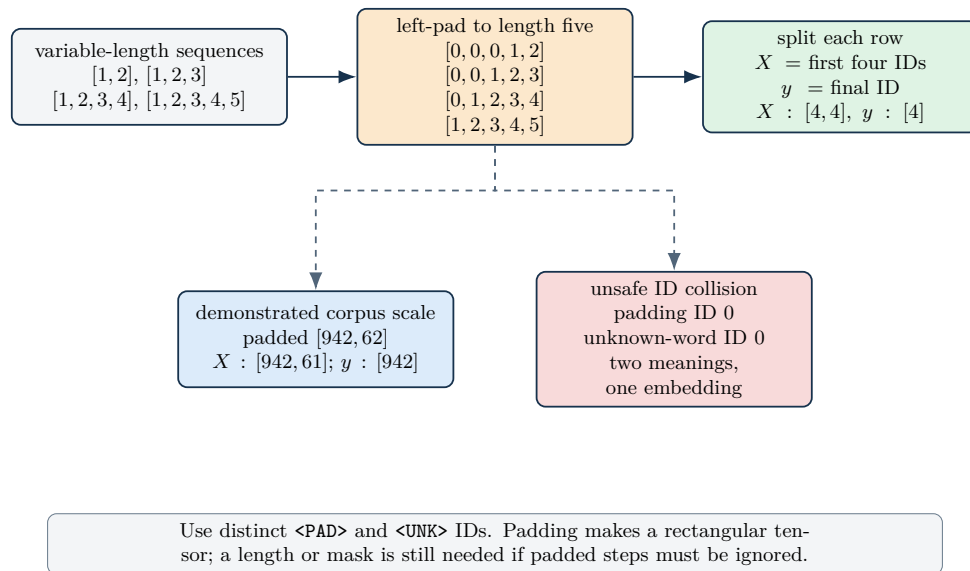


Figure 3: Variable-length prefixes become a rectangular tensor before the final column is separated as the next-token target. [Technical reference](#).

4.3 Why padding and unknown IDs must differ

Using ID 0 for both padding and unknown words gives one embedding row two incompatible meanings. A padded position means “no token here,” while an unknown token means “a real token occurred but is outside the vocabulary.” A safer mapping is

$$\langle \text{PAD} \rangle \mapsto 0, \quad \langle \text{UNK} \rangle \mapsto 1.$$

Then `nn.Embedding(..., padding_idx=0)` can keep the padding vector fixed. If recurrence should ignore padding entirely, retain true lengths and use packing or masking.

Technical precision

Left-padding keeps the final real token at the last position, which makes final-state selection convenient. Packing utilities normally expect valid tokens first and padding afterward; a production packed pipeline should right-pad and carry lengths.

5 Dataset and DataLoader contracts

5.1 Dataset responsibilities

A PyTorch `Dataset` defines the number of examples and the exact tensors returned for one index. Its invariants here are:

- X and y have the same first-axis length;
- X_i has integer shape `[61]`;
- y_i is one integer class ID;
- no mutation occurs when a sample is read.

```

from torch.utils.data import DataLoader, Dataset

class NextTokenDataset(Dataset):
    def __init__(self, inputs, targets):
        if len(inputs) != len(targets):
            raise ValueError("inputs and targets must align")
        self.inputs = inputs
        self.targets = targets

    def __len__(self):
        return self.inputs.shape[0]

    def __getitem__(self, index):
        return self.inputs[index], self.targets[index]

dataset = NextTokenDataset(X, y)
loader = DataLoader(dataset, batch_size=32, shuffle=True)

```

Listing 3: Expose prepared next-token rows through a Dataset and shuffled DataLoader

The `DataLoader` controls batching and shuffling. For batch size $B = 32$, a complete mini-batch has

$$X_b : [32, 61], \quad y_b : [32].$$

Because

$$942 = 29 \cdot 32 + 14,$$

one epoch contains 30 batches and the final batch contains 14 examples unless `drop_last=True` is set.

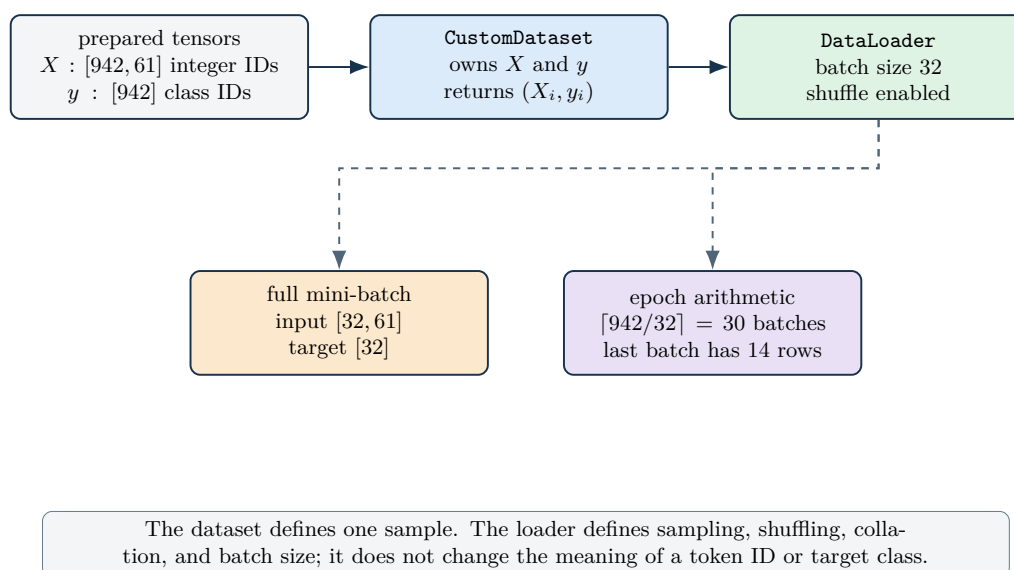


Figure 4: The Dataset defines one aligned sample; the DataLoader adds shuffle order and mini-batch axes without changing token meaning. [Technical reference](#).

Shape and state check

Cross-entropy requires one target index per input row. The correct target shape is $[B]$, not $[B, 1]$ and not $[B, V]$.

6 Embedding lookup: token IDs become vectors

6.1 Table definition and shapes

An LSTM expects continuous feature vectors, not arbitrary ID magnitudes. The embedding table is

$$E \in \mathbb{R}^{V \times D},$$

where $V = 289$ is vocabulary size and $D = 100$ is embedding width. For integer IDs

$$I \in \mathbb{N}^{B \times L},$$

with batch size B and context length $L = 61$, lookup returns

$$X = E[I] \in \mathbb{R}^{B \times 61 \times 100}.$$

The first two axes are preserved; one 100-value feature axis is added.

For a token ID j , lookup returns row $E_{j,:}$. If $e_j \in \{0, 1\}^V$ is a one-hot vector containing a single 1 at position j , then

$$e_j^\top E = E_{j,:}.$$

This equation computes the same vector as direct lookup. The lookup is more efficient because it does not materialize a length-289 mostly zero vector.

Intuition

Embedding coordinates have no predefined labels. Training changes the table so tokens useful in similar predictive contexts can acquire useful numerical relationships.

7 LSTM state and gates

7.1 Why two recurrent states are used

A plain recurrent neural network repeatedly compresses the past into one hidden vector. Long multiplication chains can make gradients vanish or explode. An LSTM introduces controlled gates and a separate cell state to improve information flow over longer intervals.

At time step t :

- $x_t \in \mathbb{R}^{B \times D}$ is the current embedded token;
- $h_{t-1} \in \mathbb{R}^{B \times H}$ is the previous hidden state exposed to the output computation;
- $c_{t-1} \in \mathbb{R}^{B \times H}$ is the previous cell state carrying gated memory;
- $H = 150$ is hidden width in this project.

PyTorch first computes four gate pre-activations together:

$$a_t = x_t W_{ih}^\top + b_{ih} + h_{t-1} W_{hh}^\top + b_{hh} \in \mathbb{R}^{B \times 4H}.$$

Here $W_{ih} \in \mathbb{R}^{4H \times D}$ maps the current input, $W_{hh} \in \mathbb{R}^{4H \times H}$ maps the previous hidden state, and $b_{ih}, b_{hh} \in \mathbb{R}^{4H}$ are learned biases. Splitting a_t into four width- H blocks gives

$$\begin{aligned} i_t &= \sigma(a_t^{(i)}), & f_t &= \sigma(a_t^{(f)}), \\ g_t &= \tanh(a_t^{(g)}), & o_t &= \sigma(a_t^{(o)}). \end{aligned}$$

The sigmoid function $\sigma(u) = 1/(1 + e^{-u})$ maps every component into $(0, 1)$. Thus i_t controls candidate insertion, f_t controls retained old cell content, and o_t controls exposed cell content. The candidate g_t lies in $(-1, 1)$.

The state updates are

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t),$$

where $\odot$ means elementwise multiplication. The first equation blends retained memory with new candidate information. The second exposes a gated, bounded view of that cell state.

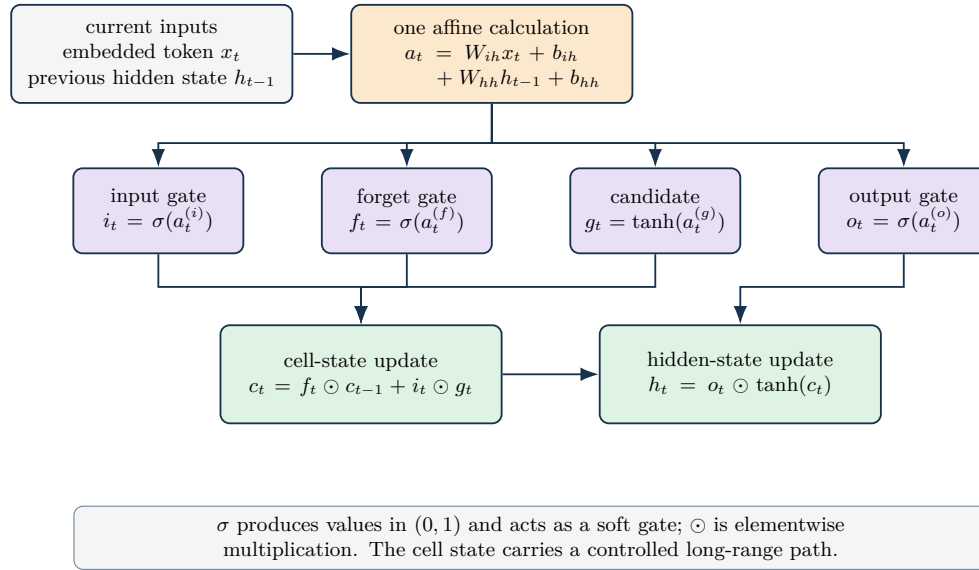


Figure 5: One LSTM step computes four gates, updates the carried cell state, and exposes a hidden state for the current time step. [Technical reference](#).

7.2 A fully expanded scalar LSTM step

Use one input feature and one hidden feature:

$$x_t = 0.6, \quad h_{t-1} = 0.2, \quad c_{t-1} = -0.1.$$

For gate order input, forget, candidate, output, choose

$$W_{ih} = \begin{bmatrix} 0.7 \\ -0.4 \\ 0.9 \\ 0.3 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 0.2 \\ 0.5 \\ -0.1 \\ 0.8 \end{bmatrix},$$

$$b_{ih} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.05 \\ 0 \end{bmatrix}, \quad b_{hh} = \begin{bmatrix} 0 \\ 0.1 \\ -0.05 \\ -0.1 \end{bmatrix}.$$

Every pre-activation can now be expanded in the same order as the compact formula:

$$\begin{aligned} a_i &= 0.7(0.6) + 0.2(0.2) + 0.1 + 0 = 0.56, \\ a_f &= -0.4(0.6) + 0.5(0.2) - 0.2 + 0.1 = -0.24, \\ a_g &= 0.9(0.6) - 0.1(0.2) + 0.05 - 0.05 = 0.52, \\ a_o &= 0.3(0.6) + 0.8(0.2) + 0 - 0.1 = 0.24. \end{aligned}$$

Activation gives

$$i_t = 0.636453, \quad f_t = 0.440286, \quad g_t = 0.477700, \quad o_t = 0.559714.$$

The new cell state is

$$\begin{aligned} c_t &= 0.440286(-0.1) + 0.636453(0.477700) \\ &= -0.044029 + 0.304034 \\ &= 0.260005. \end{aligned}$$

The new hidden state is

$$h_t = 0.559714 \tanh(0.260005) = 0.142335.$$

The positive candidate contribution exceeds the forgotten negative cell value, so the new cell state becomes positive.

```
import torch
from torch import nn

cell = nn.LSTMCell(1, 1, bias=True).double()
with torch.no_grad():
    cell.weight_ih.copy_(
        torch.tensor([[0.7], [-0.4], [0.9], [0.3]], dtype=torch.float64)
    )
    cell.weight_hh.copy_(
        torch.tensor([[0.2], [0.5], [-0.1], [0.8]], dtype=torch.float64)
    )
    cell.bias_ih.copy_(
        torch.tensor([0.1, -0.2, 0.05, 0.0], dtype=torch.float64)
    )
    cell.bias_hh.copy_(
        torch.tensor([0.0, 0.1, -0.05, -0.1], dtype=torch.float64)
    )

x_t = torch.tensor([[0.6]], dtype=torch.float64)
h_0 = torch.tensor([[0.2]], dtype=torch.float64)
c_0 = torch.tensor([[0.1]], dtype=torch.float64)
h_1, c_1 = cell(x_t, (h_0, c_0))
assert torch.allclose(c_1, torch.tensor([[0.260004751]], dtype=torch.float64))
assert torch.allclose(h_1, torch.tensor([[0.142335168]], dtype=torch.float64))
```

Listing 4: Verify the expanded scalar gate calculation against `nn.LSTMCell`

A direct PyTorch execution reproduced both numbers with maximum absolute difference exactly 0.

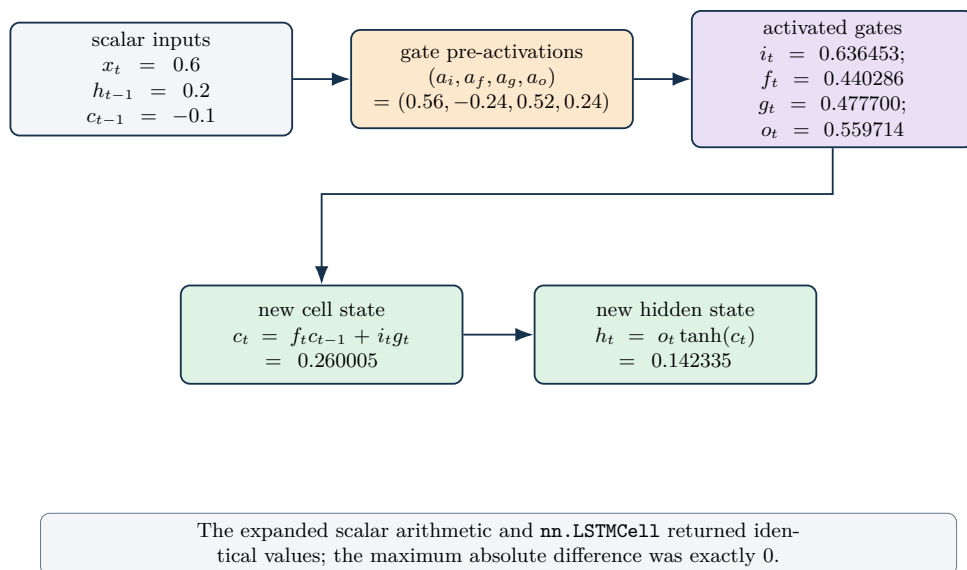


Figure 6: The scalar example follows the same affine, gate, cell, and hidden-state order as the vector LSTM equations. [Technical reference](#).

7.3 Unrolling through 61 context positions

The same LSTM parameters are reused at every context position:

$$(h_t, c_t) = \text{LSTMCell}(x_t, h_{t-1}, c_{t-1}), \quad t = 1, \dots, 61.$$

The subscript t changes the data and state, not the weight matrices. For B sequences, PyTorch performs each time step for all B rows through vectorized matrix operations.

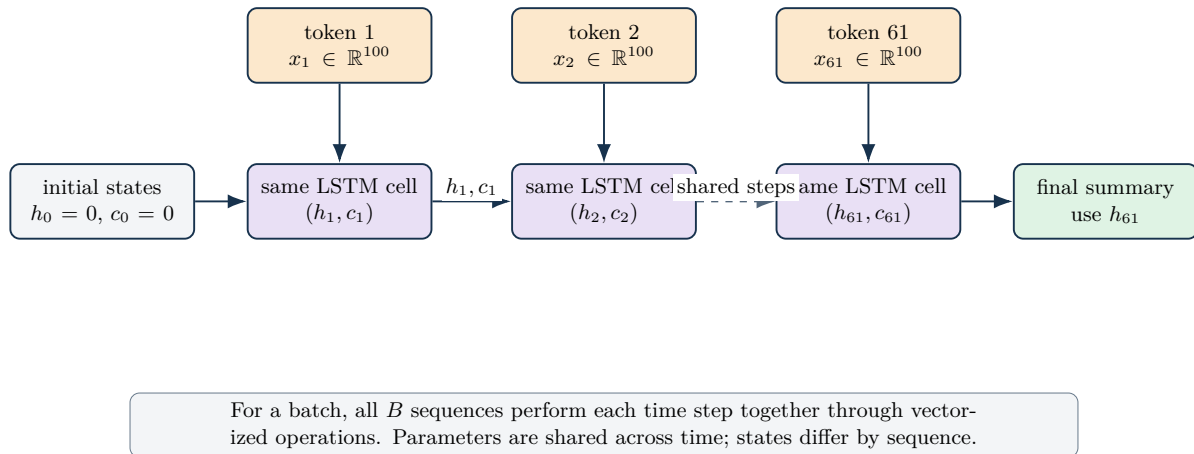


Figure 7: Unrolling exposes the state dependency across token positions while emphasizing that one parameter set is shared through time. [Technical reference](#).

Technical precision

When no initial pair is supplied, `nn.LSTM` uses zero h_0 and zero c_0 . A deterministic check compared omitted states with explicit zero tensors and found maximum absolute difference 0. Random initial states are not the default.

Pitfall

LSTM gating improves long-range gradient flow but does not guarantee perfect memory. Very long contexts, poorly scaled optimization, and irrelevant repeated padding can still degrade learning.

8 The PyTorch architecture and tensor trace

8.1 Module responsibilities

The architecture contains three trainable modules:

Embedding. Maps each of 289 IDs to 100 learned values.

LSTM. Processes 100 values per position and carries 150 hidden and cell values.

Linear classifier. Converts the final 150-value hidden state to 289 next-token logits.

```

import torch
from torch import nn

class NextWordLSTM(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, 100)
        self.lstm = nn.LSTM(
            input_size=100,
            hidden_size=150,
            batch_first=True,
        )
        self.fc = nn.Linear(150, vocab_size)

    def forward(self, token_ids):
        embedded = self.embedding(token_ids)
        all_hidden, (h_n, c_n) = self.lstm(embedded)
        final_hidden = h_n[-1]
        logits = self.fc(final_hidden)
        return logits

```

Listing 5: Define the batch-first LSTM next-token classifier

A custom `forward` method is required because the LSTM does not return one ordinary tensor. Its exact return contract is

`output, (h_n, c_n).`

There are two top-level objects. The second object is a two-tensor tuple. Treating the return value as three top-level objects is an unpacking error.

8.2 Complete batch shape trace

For $B = 32$, $L = 61$, $D = 100$, $H = 150$, one layer, one direction, and $V = 289$:

```

token IDs : [32, 61],
embedded tokens : [32, 61, 100],
all_hidden : [32, 61, 150],
    h_n : [1, 32, 150],
    c_n : [1, 32, 150],
    h_n[-1] : [32, 150],
    logits : [32, 289].

```

The leading axis of h_n and c_n combines recurrent layers and directions. It is not the sequence-length axis. Selecting `h_n[-1]` removes the layer axis while preserving batch size, including when $B = 1$.

```

model = NextWordLSTM(vocab_size=289)
batch = torch.randint(0, 289, (32, 61), dtype=torch.long)

embedded = model.embedding(batch)
all_hidden, (h_n, c_n) = model.lstm(embedded)
logits = model.fc(h_n[-1])

assert embedded.shape == (32, 61, 100)
assert all_hidden.shape == (32, 61, 150)
assert h_n.shape == (1, 32, 150)
assert c_n.shape == (1, 32, 150)
assert logits.shape == (32, 289)
assert sum(p.numel() for p in model.parameters()) == 223_739

```

Listing 6: Assert every architecture shape and count trainable parameters

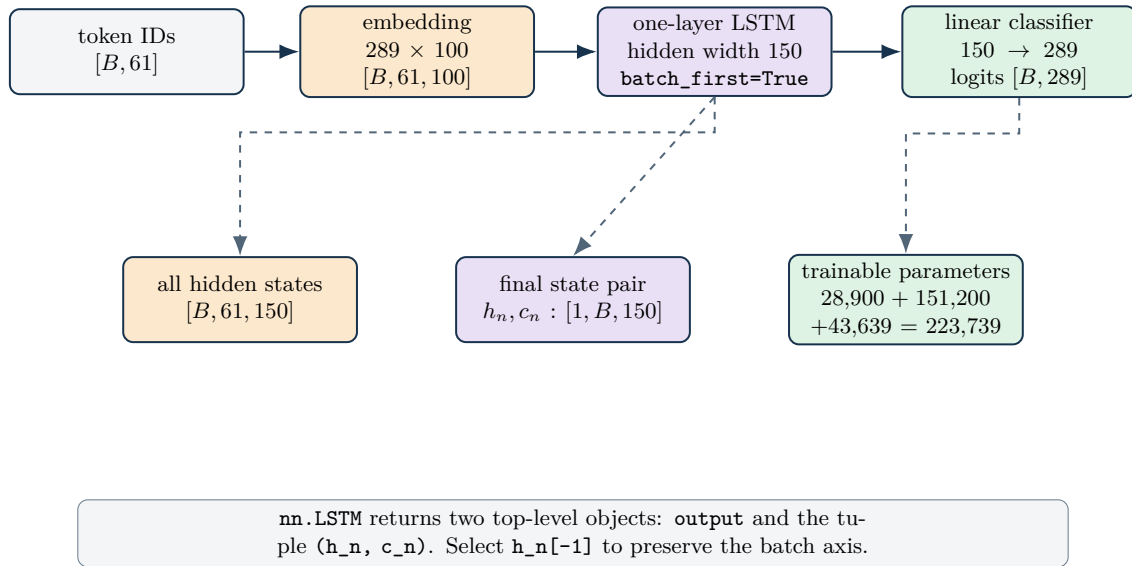


Figure 8: The project architecture preserves batch-first sequence axes, exposes both LSTM return objects, and maps the final hidden state to 289 logits. [Technical reference](#).

8.3 Parameter count

The embedding owns

$$P_{\text{emb}} = VD = 289 \cdot 100 = 28,900$$

parameters: one 100-value row per vocabulary token.

PyTorch stores four input matrices, four recurrent matrices, and two sets of four biases in one LSTM layer:

$$\begin{aligned}
 P_{\text{lstn}} &= 4HD + 4H^2 + 8H \\
 &= 4(150)(100) + 4(150)^2 + 8(150) \\
 &= 60,000 + 90,000 + 1,200 \\
 &= 151,200.
 \end{aligned}$$

The multiplier four corresponds to the input, forget, candidate, and output blocks. The $8H$ term counts both PyTorch bias vectors, each of width $4H$.

The classifier owns

$$P_{\text{fc}} = HV + V = 150 \cdot 289 + 289 = 43,639.$$

Therefore

$$P_{\text{total}} = 28,900 + 151,200 + 43,639 = 223,739.$$

A direct `numel` count matched this total.

8.4 The batch-axis failure

Without `batch_first=True`, an input shaped `[32, 61, 100]` is interpreted as

$$L = 32, \quad B = 61,$$

not 32 sequences of 61 tokens. The all-hidden tensor still has visible shape `[32, 61, 150]`, but h_n and c_n become `[1, 61, 150]`. This is a semantic bug that may not raise an exception. Shape assertions must therefore name each axis, not merely compare ranks.

9 Training objective and optimizer update

9.1 Cross-entropy for one next-token class

For one context, let $z \in \mathbb{R}^V$ contain 289 logits and let $y \in \{0, \dots, V-1\}$ be the correct next-token ID. Softmax defines

$$p_j = \frac{e^{z_j}}{\sum_{k=1}^V e^{z_k}}, \quad j = 1, \dots, V.$$

The value p_j is the normalized probability assigned to class j . Cross-entropy is

$$\ell(z, y) = -\log p_y.$$

It is small when the correct token receives high probability and large when it receives low probability. `nn.CrossEntropyLoss` accepts raw logits and performs a numerically stable log-softmax calculation internally; applying softmax before the loss is incorrect.

For a batch, the required contract is

$$z : [B, V], \quad y : [B], \quad y \text{ has integer type.}$$

9.2 Device-safe mini-batch loop

The project uses Adam, learning rate 0.001, batch size 32, and 50 epochs. Both model parameters and mini-batch tensors must live on the same device.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = NextWordLSTM(vocab_size=289).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

def train_epoch(model, loader):
    model.train()
    total_loss = 0.0
    for batch_x, batch_y in loader:
        batch_x = batch_x.to(device)
        batch_y = batch_y.to(device)

        optimizer.zero_grad(set_to_none=True)
        logits = model(batch_x)
        assert logits.shape == (batch_x.shape[0], 289)
        loss = criterion(logits, batch_y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    return total_loss / len(loader)

for epoch in range(50):
    mean_loss = train_epoch(model, loader)
    print(f"Epoch {epoch + 1}: mean loss = {mean_loss:.4f}")
```

Listing 7: Train for multiple epochs with explicit state, device, and shape handling

Each mini-batch performs six ordered operations: move tensors, clear stale gradients, compute logits, compute loss, backpropagate through all 61 recurrent positions, and update parameters. The returned value is mean mini-batch loss. If batch losses are merely summed, the printed scale also depends on the number of batches.

The project run printed the *sum* of all 30 mini-batch losses. The visible early values were

epoch	1	2	3	4
summed loss	166.6307	147.8348	135.3078	122.8751

and the final visible values were

epoch	36	37	38	39	40	41	42	43
summed loss	6.0049	5.9083	5.7534	5.7539	5.2918	5.1860	5.1703	4.9205

epoch	44	45	46	47	48	49	50
summed loss	4.8498	4.7981	4.6540	4.6560	4.5391	4.4375	4.3435

The broad decrease shows increasing fit to the derived training rows. Small increases from epoch 38 to 39 and from 46 to 47 are possible because shuffled mini-batches and Adam updates do not guarantee a monotonic value at every epoch.

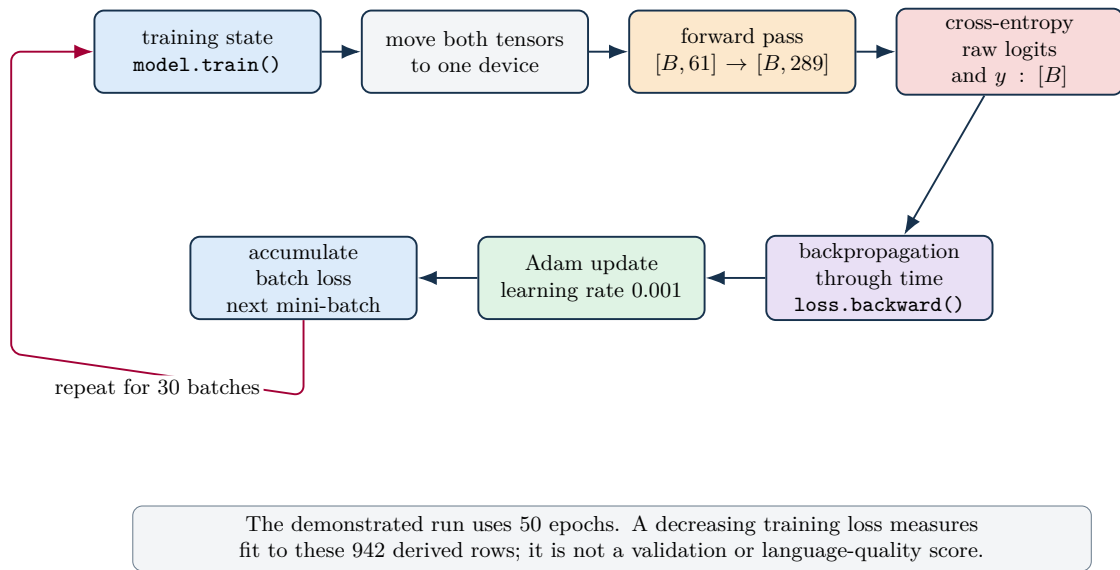


Figure 9: Training alternates batch-first recurrence, cross-entropy, backpropagation through time, and an Adam update. [Technical reference](#).

A source-shape execution on synthetic IDs performed a real backward pass and Adam update. It produced loss 5.642384, classifier-weight gradient norm 0.263077, and maximum classifier-weight change 0.001000002. These values verify mechanics and tensor flow; they are not an accuracy claim for the FAQ corpus.

Technical precision

A falling training loss only shows that 223,739 parameters are fitting 942 highly overlapping rows. Prefixes derived from the same sentence must not be split independently across training and validation sets, because that would leak nearly identical text across the boundary.

10 One-step prediction

10.1 Prediction must reproduce preprocessing

A prediction context must be lowercased, tokenized, mapped with the saved vocabulary, padded to 61 IDs, converted to `torch.long`, given a batch axis, and moved to the model's device. Contexts longer than 61 tokens need an explicit policy; retaining the most recent 61 tokens is a common fixed-window choice.


```

def encode_context(text, vocab, context_length=61, pad_id=0):
    tokens = tokenize(text)
    ids = text_to_indices(tokens, vocab)[-context_length:]
    pad_count = context_length - len(ids)
    padded = [pad_id] * pad_count + ids
    return torch.tensor([padded], dtype=torch.long)

def predict_next(model, text, vocab, id_to_token, top_k=3):
    token_ids = encode_context(text, vocab).to(device)
    model.eval()
    with torch.inference_mode():
        logits = model(token_ids)
        probabilities = logits.softmax(dim=1)
        values, indices = probabilities.topk(top_k, dim=1)

    return [
        (id_to_token[int(index)], float(value))
        for value, index in zip(values[0], indices[0])
    ]

```

Listing 8: Return the highest-probability next token and optional top-k alternatives

`model.eval()` selects evaluation behavior for modules such as dropout, while `torch.inference_mode()` disables gradient recording. Neither operation changes the learned weights. The result is a ranked list of token strings and probabilities.

The original one-token path can use `argmax` directly on logits because softmax preserves ordering:

$$\arg \max_j z_j = \arg \max_j p_j.$$

Softmax is needed only when normalized probabilities or probability-based sampling are required.

For the context `The course follows`, the exact preprocessing trace is

$$[\text{the}, \text{course}, \text{follows}] \longrightarrow [2, 6, 16] \longrightarrow [1, 61] \longrightarrow [1, 289].$$

The largest logit occurred at index 17, which maps to `a` in the saved insertion-ordered vocabulary. An explicit `id_to_token` dictionary is safer than repeatedly constructing `list(vocab.keys())`, because the inverse mapping then becomes a named, serializable contract.

Technical precision

Context width 61 is a preprocessing convention learned from the padded training rows, not a structural restriction of `nn.LSTM`. The recurrent module can accept a different sequence length, but doing so changes the input distribution. Cropping, dynamic packing, or another deliberate policy is preferable to accidental length drift.

11 Autoregressive generation

11.1 Repeated one-step prediction

Autoregressive generation appends a predicted token and then treats the longer string as the next context. If $\hat{w}_{t+1}$ is the selected token, the update is

$$(w_1, \dots, w_t) \longrightarrow (w_1, \dots, w_t, \hat{w}_{t+1}).$$

Repeating this update produces several new words.

```
def generate(model, prompt, vocab, id_to_token, new_tokens=10):
    words = tokenize(prompt)
    for _ in range(new_tokens):
        candidates = predict_next(
            model,
            " ".join(words),
            vocab,
            id_to_token,
            top_k=1,
        )
        next_token, _ = candidates[0]
        words.append(next_token)
    return " ".join(words)
```

Listing 9: Generate several words with deterministic greedy decoding

Greedy decoding always selects the highest-probability token. It is deterministic for fixed weights and input, but one early mistake changes every later context. A fixed number of generation steps is also necessary when no end-of-sequence token is defined.

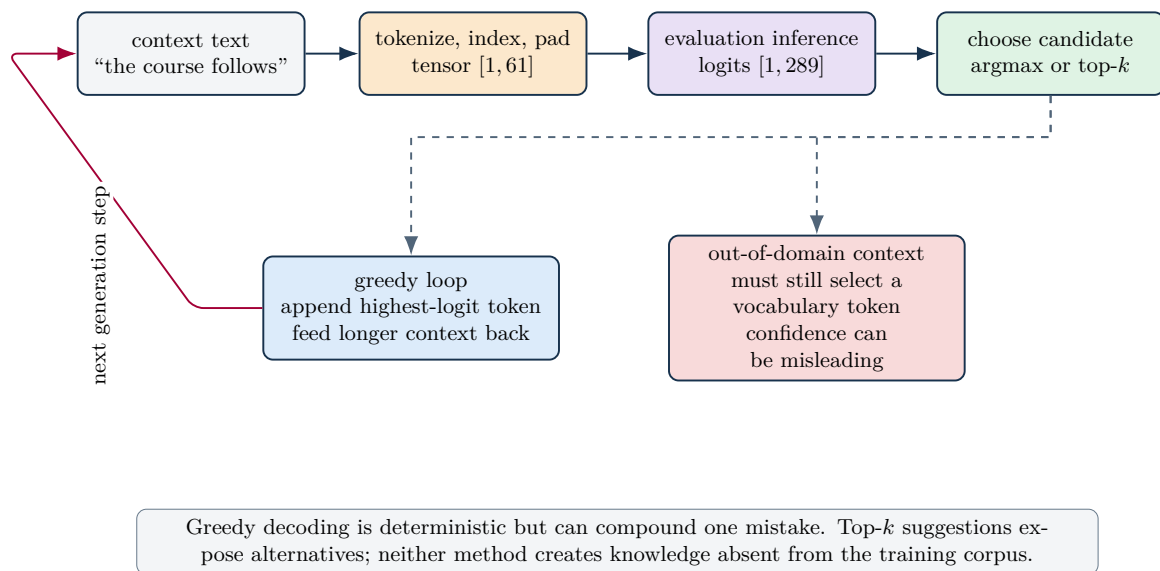


Figure 10: Prediction converts one context to 289 logits, selects either one greedy token or several top candidates, and optionally feeds a selected token back into the context. [Technical reference](#).

A deterministic tiny-corpus smoke test learned six prefix rows and reproduced

the course follows a monthly subscription model.

It reached training accuracy 1.0 and loss 0.004668 after 27 epochs. At successive generation steps, the selected tokens `a`, `monthly`, `subscription`, and `model` received probabilities 0.997871, 0.998437, 0.992477, and 0.993605. These values show that the implementation can memorize a tiny sequence; they do not measure generalization.

For the unrelated context `hello how are`, the same tiny model returned `follows` with probability 0.991755. High confidence therefore does not imply that the context is supported by the corpus.

The FAQ-fitted model produced three informative greedy traces:

- the course follows a monthly continued with subscription model, then where you have to make monthly payments;
- you can check the continued with detailed syllabus here and then began reproducing a URL-like fragment;
- the unrelated prompt `hi how are` continued with `i join the program late, can i still ask`.

The first two traces copy locally consistent FAQ patterns. The third bends an unsupported prompt toward a memorized enrolment sentence, exposing the absence of an out-of-domain rejection mechanism. A call to `time.sleep(0.5)` changes only display pacing; it has no effect on model probabilities.

11.2 Top-k suggestions and sampling

A keyboard commonly presents several candidates rather than one. `torch.topk` returns the k largest values and indices. If diversity is desired, sample from a temperature-adjusted distribution:

$$p_j^{(\tau)} = \frac{\exp(z_j/\tau)}{\sum_k \exp(z_k/\tau)}, \quad \tau > 0.$$

The temperature τ rescales logits before softmax. Values below 1 sharpen the distribution; values above 1 flatten it. Sampling can increase variety but cannot create reliable facts absent from training.

Pitfall

Calling a one-step function repeatedly without truncating context eventually exceeds the 61-token input contract. Retain the most recent 61 IDs or use an architecture designed for a different context policy.

12 Padding-aware production design

12.1 Right padding and true lengths

The fixed left-padded project tensor is easy to inspect, but it makes the LSTM process many padding positions. A scalable design uses distinct padding and unknown IDs, right-pads each mini-batch only to its local maximum length, and passes true lengths to `pack_padded_sequence`.

```
from torch.nn.utils.rnn import pack_padded_sequence

class PackedNextWordLSTM(nn.Module):
    def __init__(self, vocab_size, pad_id=0):
        super().__init__()
        self.embedding = nn.Embedding(
            vocab_size,
            embedding_dim=100,
            padding_idx=pad_id,
        )
        self.lstm = nn.LSTM(100, 150, batch_first=True)
        self.fc = nn.Linear(150, vocab_size)

    def forward(self, token_ids, lengths):
        embedded = self.embedding(token_ids)
        packed = pack_padded_sequence(
            embedded,
            lengths.cpu(),
            batch_first=True,
            enforce_sorted=False,
        )
        _, (h_n, _) = self.lstm(packed)
        return self.fc(h_n[-1])
```

Listing 10: Ignore right-padding steps with a packed LSTM representation

The token tensor has shape $[B, L_{\max}]$ and the length vector has shape $[B]$. Packing prevents padded positions from becoming recurrent steps. The classifier still returns $[B, V]$ logits.

12.2 Why the saved artifact is more than model weights

A reusable predictor needs a bundle containing

- model state dictionary and architecture hyperparameters;
- token-to-ID and ID-to-token mappings;
- padding, unknown, and optional end-token IDs;
- normalization and tokenization configuration;
- context length and padding direction;
- package versions and, when reproducibility matters, random seeds.

Weights without these mappings cannot interpret input IDs or output indices correctly.

13 Failure modes and disciplined repairs

Failure	Observable symptom	Structural repair
Floating token IDs Target shape $[B, 1]$	embedding rejects a float tensor cross-entropy reports a multi-target error	use <code>torch.long</code> IDs reshape to one-dimensional $[B]$
ID equal to V Unpadded variable rows Three-way LSTM unpack	embedding raises an index error default collation cannot stack tensors Python receives only two top-level objects	map unseen tokens to <code><UNK></code> pad in a custom collation function use <code>output, (h_n, c_n)</code>
Wrong axis convention Unrestricted <code>squeeze()</code> Padding equals unknown	h_n uses sequence length as batch size batch axis disappears at $B = 1$ one embedding row receives two meanings	set and assert <code>batch_first=True</code> select <code>h_n[-1]</code> reserve distinct special IDs
Tensor remains on CPU	device-mismatch exception during GPU work	assign both <code>tensor.to(device)</code> results
Softmax before loss No evaluation state Long generation context Training loss treated as quality	objective receives transformed scores future dropout behaves as training input exceeds width 61 memorization is called generalization	pass raw logits to cross-entropy call <code>eval()</code> and disable gradients crop or redesign the context policy evaluate held-out sentences and prefixes

Five failure contracts were reproduced directly: floating embedding IDs, rank-two class targets, an out-of-range vocabulary ID, unequal-length default collation, and incorrect LSTM unpacking. Executed failures are useful specifications because they state exactly what each API refuses.

13.1 Evaluation beyond training loss

Split complete source documents or sentences before generating prefixes so related prefixes remain in one partition. On held-out text, report mean cross-entropy and perplexity:

$$\text{PPL} = \exp\left(\frac{1}{N} \sum_{i=1}^N \ell_i\right),$$

where N is the number of held-out targets and ℓ_i is cross-entropy for target i . Perplexity is the exponential of average surprise; lower is better only when tokenization and evaluation data are held fixed.

Also measure top-1 accuracy, top- k accuracy, behavior on out-of-domain contexts, latency, and memory use. For suggestion systems, a useful candidate set can matter more than one greedy continuation.

13.2 Engineering improvements

1. Use a tested sentence tokenizer and define punctuation policy explicitly.
2. Replace word-only IDs with subword tokens to reduce unknown words.
3. Separate <PAD>, <UNK>, and <EOS> meanings.
4. Right-pad local mini-batches and pack true lengths.
5. Clip gradients when long sequences cause unstable norms.
6. Add validation-based early stopping instead of fixing 50 epochs blindly.
7. Compare LSTM baselines with GRUs and transformer decoders.
8. Inspect top- k calibration rather than treating a large softmax value as knowledge.
9. Train only on text whose privacy, licensing, and quality are appropriate for the application.

Takeaway

Most defects are contract defects: inconsistent token IDs, wrong padding semantics, an axis swap, invalid target shape, incorrect return unpacking, device mismatch, or an unsupported evaluation claim. Explicit invariants prevent these errors earlier than loss debugging.

14 Review questions and exercises

1. What probability distribution does a next-word model estimate for a prefix $w_1, \dots, w_t$?
2. Why does a sentence of n tokens produce $n - 1$ nonempty prefix-target rows?
3. Recreate all four rows for `python libraries for data science`.
4. Explain why vocabulary ID 8 is not numerically more meaningful than ID 2.
5. Why must <PAD> and <UNK> use different IDs?
6. Derive the $[942, 62] \rightarrow [942, 61]$ and $[942]$ split.
7. For batch size 32, why are there 30 batches and 14 rows in the last batch?
8. State every axis in an embedded tensor shaped $[32, 61, 100]$.
9. Define the input, forget, candidate, and output gates in plain formal language.
10. Recompute $a_i = 0.56$ and $c_t = 0.260005$ in the scalar example.
11. Why does `nn.LSTM` return two top-level objects rather than three?
12. Derive all 151,200 LSTM parameters from $D = 100$ and $H = 150$.
13. How does `batch_first=False` silently reinterpret $[32, 61, 100]$?
14. Why does cross-entropy require logits $[B, V]$ and integer targets $[B]$?
15. Distinguish training loss, held-out cross-entropy, perplexity, and top- k accuracy.
16. Why can greedy generation compound one wrong token?
17. Design a test that exposes overconfidence on an unrelated context.
18. Modify the system to stop at an end token instead of a fixed word count.
19. Implement a right-padding collation function that returns token IDs, lengths, and targets.
20. Explain why model weights are unusable without the exact saved vocabulary and tokenizer.

Takeaway

A reliable LSTM language-model pipeline begins with a precise token and target definition, turns it into explicit tensor shapes, verifies every gate and axis, trains on raw logits, reproduces preprocessing at inference, and evaluates on held-out text. The recurrent layer is important, but data construction and interface contracts determine what the system can actually learn and return.